Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Центр обучения проектированию и разработке игр

Выпускная квалификационная работа бакалавра

Оптимизация GPU-Реализации XPBI для симуляции пластических деформаций в реальном времени

Автор:
Студент группы Б05-222
Кирьянов Григорий Алексеевич

Научный руководитель:
Протасов Антон Андреевич



Москва 2026

Аннотация

Оптимизация GPU-Реализации XPBI для симуляции пластических деформаций в реальном времени
Кирьянов Григорий Алексеевич

В работе рассматривается задача повышения производительности алгоритма XPBI для моделирования деформируемых материалов в физическом движке реального времени. Актуальность исследования обусловлена тем, что исходный метод, обладая устойчивостью и поддержкой неупругих деформаций, остаётся вычислительно затратным при GPU-реализации. Целью работы является разработка модификации XPBI, уменьшающей стоимость шага симуляции без потери качественного поведения материала. Для достижения этой цели предложена схема, объединяющая многоуровневый солвер, иерархическую структуру соседства на основе триангуляции Делоне и двухэтапный солвер GS2. Алгоритм реализован на связке C# и HLSL с выполнением основных вычислений на графическом ускорителе. Проведено экспериментальное сравнение исходного и модифицированного вариантов на наборе тестовых сцен с 10 000 частиц. Результаты показывают, что предложенная модификация обеспечивает ускорение до 2.6 раза по сравнению с исходной реализацией XPBI, сохраняя устойчивость вычислений, корректную обработку контактов и визуальную правдоподобность деформаций.

Abstract

Оптимизация GPU-Реализации XPBI для симуляции пластических деформаций в реальном времени

Содержание

1 Введение **4**

2 Определения и обозначения **6**

3 Обзор и анализ алгоритма XPBI **7**
3.1 Семейство алгоритмов PBD 7
3.2 Схема алгоритма XPBI 8
3.3 Псевдокод шага алгоритма XPBI 8
3.4 Алгоритмические компоненты алгоритма XPBI 9
3.4.1 Поиск соседей 9
3.4.2 Солвер системы ограничений 10

4 Обзор альтернативных алгоритмов **11**
4.1 Обзор методов поиска соседей 11
4.2 Обзор солверов для системы ограничений 12

5 Построение решения задачи **15**
5.1 Многоуровневый солвер 15
5.1.1 Перенос характеристик частиц 15
5.1.2 Шаг солвера 15
5.1.3 Столкновения и закрепления 15
5.2 Поиск соседей 16
5.2.1 Иерархия Делоне 16
5.2.2 Построение облака точек 16
5.2.3 Запрос соседей в радиусе 17
5.2.4 Параллелизация солвера 17
5.2.5 3D случай 17
5.3 Выбор солвера 17
5.4 Псевдокод видоизменённого шага алгоритма XPBI 18

6 Детали реализации **20**
6.1 Организация данных 20
6.2 Поддержка геометрии и отношений соседства 20
6.3 Набор ограничений 21
6.4 Организация солверов 21

7 Результаты **22**
7.1 Конфигурация тестирования 22
7.2 Результаты сравнения вариантов XPBI 23
7.3 Результаты сравнения структур поиска соседей 24
7.4 Выводы по экспериментам 25

8 Заключение **26**

1 Введение

В современных физических движках численные методы для моделирования деформаций должны быть не только физически правдоподобными и устойчивыми, но и достаточно эффективными для обеспечения вычислений в реальном времени. Существенный интерес представляют методы, в которых взаимодействие между материальными точками задаётся не фиксированной сеточной топологией, а динамически перестраиваемой структурой соседства. В отличие от классических сеточных постановок, такой подход позволяет описывать сценарии с пластическими деформациями и изменением внутренней структуры материала. Это делает подобные методы перспективными для физических движков общего назначения, в которых требуется поддержка широкого спектра материалов в рамках единой вычислительной схемы.

Одним из таких методов является XPBI, предложенный Yu и соавт. в 2024 году, который развивает идеи семейства PBD и XPBD и сочетает позиционно-ориентированную постановку ограничений с динамически определяемыми окрестностями частиц. Данный подход позволяет моделировать неупругие деформации без жёсткой привязки к фиксированной топологии, однако этот алгоритм предложен недавно и его практическое применение в составе игрового движка требует дополнительного исследования и оптимизации отдельных компонентов. [1]

Метод XPBI — это эффективный подход к GPU-моделированию неупругих деформируемых материалов в реальном времени, но практическое применение этого метода в составе физического движка по-прежнему предъявляет высокие требования к производительности. Полная стоимость шага симуляции в исходной схеме остаётся существенной, что ограничивает масштаб моделируемых сцен и запас вычислительных ресурсов для сопутствующих подсистем движка. Поэтому актуальной является задача дальнейшей оптимизации отдельных компонентов XPBI, направленной на уменьшение времени шага симуляции без нарушения устойчивости вычислений и корректности моделирования деформаций.

Целью данной работы является исследование и разработка модификации алгоритма XPBI, обеспечивающей более производительную симуляцию деформируемых материалов на графическом ускорителе в контексте игрового движка.

Постановка задачи: Рассматривается система материальных частиц, описывающих набор деформируемых тел, для которых на каждом шаге времени необходимо сформировать и решить систему ограничений этой симуляции и получить следующее состояние системы за время, допустимое для вычислений в реальном времени, и с визуально достоверной точностью. При этом требуется уменьшить полную стоимость шага симуляции по сравнению с исходным алгоритмом XPBI, не нарушая его ключевых свойств: устойчивости вычислений, поддержки неупругих деформаций, корректной обработки контактов и применимости в GPU-реализации. [1]

Для достижения поставленной задачи в работе необходимо выполнить следующие шаги:

- проанализировать структуру алгоритма XPBI и выделить его основные вычислительные узкие места;
- исследовать альтернативные подходы к реализации этих компонентов, пригодные для рассматриваемой задачи;
- разработать модифицированную схему XPBI;
- реализовать исходный и модифицированный варианты алгоритма на GPU;

- провести сравнительный анализ исходного и предложенного вариантов на наборе тестовых сценариев и оценить влияние отдельных оптимизаций на производительность и качество симуляции.

Научная новизна работы состоит в адаптации многоуровневого подхода, иерархической структуры соседства и двухэтапной схемы решения ограничений к задаче ускорения XPBI на GPU, а также в исследовании их совместного применения в контексте физического движка реального времени.

Во второй главе рассматриваются базовые принципы XPBI и анализируются его ограничения. В третьей главе проводится обзор альтернативных алгоритмов для поиска соседей и решения системы ограничений. В четвёртой главе предлагается модифицированная схема алгоритма. В пятой главе описываются детали её реализации на CPU и GPU. Далее приводятся результаты экспериментального сравнения и формулируются основные выводы работы.

Реализация предложенного алгоритма доступна в репозитории GitHub [2].

2 Определения и обозначения

PBD (Position-Based Dynamics) Класс позиционно-ориентированных методов физической симуляции, в которых на каждом шаге сначала вычисляется предсказанное состояние системы, а затем положения частиц итерационно корректируются проекцией ограничений.[3]

XPBD (eXtended Position-Based Dynamics) Расширение PBD, вводящее жёсткость и накопленные множители Лагранжа, что позволяет задавать физически осмысленную податливость ограничений, существенно ослабляет зависимость результата от шага по времени и числа итераций и даёт оценки сил ограничений.[4]

XPBI (eXtended Position-based Inelasticity) Расширение XPBD, предназначенное для моделирования континуальной неупругости в безсеточном частичном представлении. В XPBI деформационный градиент отслеживается через градиент скорости, а пластический отклик учитывается неявно непосредственно внутри XPBD-итераций.[1]

Ограничение Условие, накладываемое на степени свободы системы и определяющее допустимые состояния или допустимые изменения её конфигурации. В PBD-методах ограничение задаётся в виде функции от положений, скоростей или внутренних переменных и удовлетворяется за счёт итеративных поправок.

Материальное ограничение Ограничение, описывающее локальный отклик материала через меру деформации, связанную с частицей и её окрестностью, и задающее вклад упругого, пластического и объёмного отклика.

Многоуровневый солвер Схема решения, в которой задача рассматривается на нескольких вложенных уровнях разрешения; грубые уровни ускоряют распространение низкочастотных поправок и уменьшают число итераций локального солвера.

Раскрашенный метод Гаусса–Зейделя Параллельная реализация метода Гаусса–Зейделя, в которой ограничения или узлы разбиваются на независимые группы. Элементы одной группы могут обрабатываться одновременно, а переход между группами сохраняет последовательное распространение поправок.

GS2 (Двухэтапный метод Гаусса–Зейделя) Вариант метода Гаусса–Зейделя для решения разреженных систем, в котором точное решение треугольной подзадачи заменяется фиксированным числом массивно параллельных внутренних итераций метода Якоби-Ричардсона.[5]

Иерархия Делоне Последовательность вложенных триангуляций Делоне, построенных на вложенных подмножествах исходного множества точек и используемых для поиска соседей, межуровневых переходов и организации многоуровневого решения.[6]

Родительские вершины Вершины более грубого уровня, с которыми связывается вершина для интерполяции состояния и переноса межуровневых поправок.

XSPH-сглаживание Эвристическая процедура коррекции скорости частицы по скоростям соседей, уменьшающая численный шум и высокочастотные осцилляции.

3 Обзор и анализ алгоритма XPBI

3.1 Семейство алгоритмов PBD

Широко применяемым в контексте вычислений в реальном времени методом является Position-Based Dynamics (PBD), предложенный Мюллером и соавт. [3], который получил распространение благодаря вычислительной простоте и высокой численной стабильности. Классический PBD, однако, обладает рядом существенных ограничений: он затрудняет моделирование нежестких ограничений и реалистичную передачу упругой энергии во времени.[7]

Для снятия этих ограничений Macklin, Müller и Chentanez (2016) предложили eXtended Position-Based Dynamics (XPBD), позволяющий явно управлять жесткостью ограничений, переносить силу ограничения между итерациями через множители Лагранжа и корректно моделировать более широкий спектр материалов.[7]

Тем не менее XPBD, как и исходный PBD, опирается на фиксированную структуру (сетку) частиц объекта. При такой формулировке естественным образом описываются только упругие деформации, тогда как любая долговременная пластическая деформация потребовала бы перестройки или обновления этой структуры, что существенно осложняет моделирование. В ответ на это Yu и соавт. (2024) предложили метод eXtended Position-Based Inelasticity (XPBI), вводящий модифицированные формулировки ограничений и алгоритмы релаксации, которые обеспечивают устойчивое моделирование неупругих деформаций в рамках парадигмы PBD. [1]

Помимо семейства PBD, для моделирования деформируемых сред широко применяются также FEM, MPM, Projective Dynamics и VBD. FEM и MPM обычно рассматриваются как более физически выразительные подходы и хорошо подходят для задач, где важна физическая точность описания поведения среды, однако это сопровождается очень высокой вычислительной стоимостью, что делает невозможным их использование в риалтайм симуляциях.[8] Projective Dynamics и VBD, часто обеспечивают более высокую устойчивость и хорошую производительность в своих классах задач, чем PBD-методы, но гораздо менее удобны в сценариях с жёсткими, сильно неоднородными ограничениями, характерными для физических движков общего назначения. [9] [10]

В связи с вышеуказанными ограничениями других методов, именно XPBI был выбран основным методом данного исследования как предоставляющий оптимальный компромисс для использования в физических движках, так как при достаточно высокой эффективности для вычислений в реальном времени он поддерживает наиболее широкий спектр материалов и поддерживает пластичные деформации, что расширяет возможности реализуемых взаимодействий внутри движка.

3.2 Схема алгоритма XPBI

В методе XPBI деформируемое тело представляется набором материальных частиц. Каждая частица хранит массу, положение, скорость, параметры материала, а также переменные, описывающие её текущее деформационное состояние. [1]

В отличие от сеточных методов, связь между частицами не задаётся постоянной топологией. Локальное взаимодействие определяется через текущие окрестности частиц некоторого радиуса и сглаживающие ядра. Это означает, что пары взаимодействующих частиц, а также их вклад в вычисление локальных величин определяются динамически, а не заранее заданными рёбрами или элементами. [1]

На каждом шаге алгоритма, после обновления текущей скорости частиц с учётом внешних сил, для каждой частицы строится список соседей, находящихся в пределах радиуса взаимодействия. После этого по найденной окрестности, используя сглаживающие ядра Вендланда, вычисляется локальная аппроксимация градиента скорости

$$\nabla \mathbf{v}_i = \sum_{j \in \mathcal{N}_i} V_j \left(\mathbf{v}_j - \mathbf{v}_i\right) \otimes L_i \nabla \hat{W}_{ij}, \tag{1}$$

где V_j — объём, ассоциированный с частицей j, $\hat{W}_{ij}$ — значение ядра, L_i - коррекция, связанная с несимметричностью набора соседей. Полученный градиент используется для обновления деформационного состояния материала [1]

$$\mathbf{F}_i^{n+1} = \left(\mathbf{I} + \Delta t \, \nabla \mathbf{v}_i\right) \mathbf{F}_i^n. \tag{2}$$

Полученная локальная оценка деформации служит основой для построения континуальных ограничений, описывающих механическое поведение среды. Такие ограничения могут отвечать за сохранение объёма, сопротивление растяжению, сдвигу и т.д. Например, основное ограничение энергии деформации $\mathbf{C}_i(\mathbf{F}_i) = 0$ формулируется как:

$$\frac{1}{2\alpha_i} C_i(\mathbf{F}_i)^2 = \Phi_i = V_i^0 \, \Psi(\mathbf{F}_i), \rightarrow C_i(\mathbf{F}_i) = \sqrt{2 \, \Psi(\mathbf{F}_i)}. \tag{3}$$

где Φ_i — полная энергия деформации частицы, $\Psi(\mathbf{F}_i)$ — плотность энергии деформации (формула, зависящая от модели упругости), V_i^0 — начальный объём, ассоциированный с частицей, а α_i — параметр жёсткости. [1]

Построенная система ограничений решается итеративно в духе XPBD: для каждого ограничения на каждой итерации вычисляются его текущее нарушение, градиенты по степеням свободы участвующих частиц и приращение соответствующего множителя Лагранжа с учётом параметра жёсткости. [7] Затем по найденному приращению выполняется локальная коррекция скоростей (в отличие от XPBD, система решается относительно скорости, а не положения частицы), уменьшающая нарушение ограничения; накопление множителей Лагранжа между итерациями позволяет учитывать силу коррекции ограничения на следующих итерациях. [1]

Для моделирования неупругости внутри каждой итерации выделяется пластическая и упругая составляющая деформации. Также используются несколько дополнительных эвристик (коррекция позиций, XSPH-сглаживание) для получения более равномерной плотности точек и стабильности алгоритма. [11] [1]

3.3 Псевдокод шага алгоритма XPBI

Обозначения. Пусть L — множество всех частиц системы. Через $\mathcal{N}(a)$ обозначим множество соседей частицы a, найденных внутри радиуса поддержки, а через χ — раскраску ячеек фоновой пространственной сетки, используемую в раскрашенном методе Гаусса–Зейделя, N_{GS} - число итераций метода Гаусса–Зейделя.

Алгоритм 1 Один шаг XPBI-солвера [1]

Входные данные: Состояние системы на шаге n, шаг времени Δt, множество частиц L

Результат: Обновлённое состояние частиц множества L

Построить множества соседей $\mathcal{N}(a)$ для всех частиц $a \in L$ ▷ по равномерной фоновой сетке и радиусу поддержки
Применить внешние силы и получить предсказанные скорости
Вычислить матрицы коррекции градиента для всех частиц
Построить контакты и закрепления
Построить по ним жёсткие ограничения
Обнулить множители Лагранжа ограничений
Построить раскраску χ фоновой сетки
for $s = 1$ **to** N_{GS} **do**
    for all цветов $c \in \chi$ **do**
        parallel for each ячейка сетки Q цвета c
        for all частиц $a \in Q$ **do**
            Оценить градиент скорости частицы a по её соседям $\mathcal{N}(a)$
            Вычислить пробное деформационное состояние частицы a
            Выделить упругую часть деформации возвратным отображением
            Вычислить значение материального ограничения частицы a и его градиенты
            Обновить множитель Лагранжа частицы a
            Обновить скорости частицы a и её соседей по алгоритму Гаусса–Зейделя
        end for
    end for
    Выполнить один проход решения жёстких ограничений
    Выполнить коррекцию положений частиц
end for
Выполнить XSPH-сглаживание скоростей
Зафиксировать новое деформационное состояние частиц
Интегрировать положения частиц с использованием полученных скоростей

3.4 Алгоритмические компоненты алгоритма XPBI

В алгоритме XPBI 2 компонента представляют особый интерес с точки зрения производительности - алгоритм поиска соседей и алгоритм решения системы ограничений, так как остальные шаги выполняются за константное число операций на каждой итерации (в предположении ограниченного числа соседей частицы).

3.4.1 Поиск соседей

Так как частицы в ходе моделирования перемещаются, их локальные окрестности изменяются на каждом шаге времени. По этой причине списки соседей не могут быть заданы заранее один раз на всё моделирование и должны регулярно перестраиваться.

В оригинальной работе для поиска соседей используется равномерная пространственная сетка. Все частицы раскладываются по её ячейкам в соответствии с их текущими координатами. После этого для каждой частицы соседи ищутся не среди всех частиц системы, а только среди частиц, попавших в ту же ячейку и в соседние.

Несмотря на простоту реализации и параллелизации, поиск соседей на основе равномерной пространственной сетки имеет ряд существенных недостатков:

- Такая схема, как правило, требует полной перестройки сетки на каждом шаге моделирования, так как частицы непрерывно перемещаются, и их принадлежность ячейкам меняется. Даже в тех случаях, когда за один шаг времени частицы смещаются незначительно и локальная структура окрестностей почти не меняется, эти операции всё равно выполняются полностью.

- Равномерная сетка плохо использует однородность движения. В большинстве симуляций за один шаг времени лишь небольшая часть частиц существенно меняет набор частиц в окрестности, однако традиционная схема не использует это свойство и пересчитывает структуру соседства для всех частиц одинаково. Это приводит к избыточной работе и снижает общую эффективность алгоритма.

- Равномерная сетка плохо адаптируется к неравномерному распределению частиц. В плотных областях отдельные ячейки оказываются перегруженными, тогда как в разреженных значительная часть сетки практически не используется.

3.4.2 Солвер системы ограничений

Для решения системы ограничений в XPBI используется разноцветный метод Гаусса–Зейделя. Его цель состоит в том, чтобы сохранить характерную для схемы Гаусса–Зейделя быструю передачу обновлённой информации между зависимыми степенями свободы, но при этом выделить группы независимых операций, пригодные для параллельного выполнения. Для этого рассматривается текущий граф зависимостей вычислительной схемы. Вершины этого графа соответствуют выбранным единицам обработки, например частицам или пространственным ячейкам, а ребро проводится между двумя вершинами тогда, когда соответствующие обновления нельзя выполнять одновременно из-за конфликта записи или непосредственной вычислительной зависимости.

Раскраска строится так, чтобы любые две смежные вершины имели разные цвета. Тогда все вершины одного цвета образуют независимое множество и могут обрабатываться параллельно в рамках одного цветового прохода без нарушения корректности вычислений. Одна итерация солвера выполняется как последовательность проходов по цветам. После завершения обработки очередного цвета алгоритм переходит к следующему, благодаря чему обновления, полученные на предыдущих проходах, могут сразу использоваться в последующих, как и в обычной схеме Гаусса–Зейделя.

Подобный подход является компромиссом между полностью параллельной схемой Якоби и полностью последовательным методом Гаусса–Зейделя. С одной стороны, он позволяет распараллелить часть вычислений; с другой стороны, степень параллелизма ограничивается числом цветов и размером отдельных цветовых групп, а между цветовыми проходами требуется синхронизация.

Основным недостатком алгоритма является то, что степень параллелизма ограничивается размером текущей цветовой группы: если граф взаимодействий раскрашен в C цветов, то вместо одновременной обработки всех активных частиц на каждом подшаге фактически обрабатывается лишь одна из C независимых подгрупп. Для GPU это приводит к недозагрузке вычислительных блоков, снижению заполненности и ухудшению задержек памяти, особенно если отдельные цветовые множества малы. Кроме того, обработка по цветам требует глобальной синхронизации между цветовыми проходами и увеличивает число dispatch-вызовов, что создаёт дополнительные накладные расходы.

4 Обзор альтернативных алгоритмов

4.1 Обзор методов поиска соседей

Применимо к XPBI, поиск ближайших соседей должен обладать следующими свойствами:

- быть динамическим - каждый шаг алгоритма точки сдвигаются
- быстро обновляться при малых сдвигах
- учитывать однородность движения частиц (так как объекты могут часто двигаться без деформации, а многие деформации, например изгиб, сильно локализованы)
- быть массивно параллелизуемым на GPU

Поэтому ряд широко известных структур, например quadtree, octree, kd-деревья и иные иерархические деревья, не рассматриваются, так как неэффективны в динамике.

Клеточные методы. В клеточном методе расчётная область разбивается на ячейки размера порядка радиуса поддержки ядра, и каждая частица помещается в ячейку по своему центру. Тогда поиск соседей выполняется только в текущей и соседних ячейках: не более 9 в двумерном случае и 27 в трёхмерном. Построение такой структуры стоит $O(n)$, а при ограниченной локальной плотности полный проход по всем частицам на практике ведёт себя как $O(nk)$, где k — среднее число истинных соседей.

Структуру можно полностью перестраивать на каждом шаге за $O(n)$, а при инкрементальном обновлении стоимость определяется только числом частиц, сменивших ячейку. Их основной недостаток связан с большой скрытой константой: приходится проверять много лишних кандидатов из соседних ячеек, особенно в трёхмерном случае. Кроме того, плотная равномерная сетка неэкономно хранит пустое пространство, а пространственное хеширование уменьшает расход памяти ценой ухудшения локальности доступа.

Списки Верле развивают клеточный подход за счёт использования временной когерентности. Для каждой частицы строится расширенный список кандидатов в радиусе $h + \Delta$, где h — физический радиус взаимодействия, а Δ — запас безопасности. Полная перестройка списка обычно выполняется с помощью клеточной структуры и стоит порядка $O(nk)$ для всей системы, но требуется не на каждом шаге, а только после того, как смещения частиц превысят допустимый порог. При малых смещениях между временными слоями это даёт заметное снижение средней стоимости поиска. [12]

Недостатки списков Верле состоят в дополнительной памяти порядка $O(nk)$, наличии ложных кандидатов внутри расширенного списка и сильной зависимости эффективности от выбора delta: малое значение вызывает частые перестройки, большое — увеличивает число лишних проверок.

Триангуляция Делоне. В отличие от клеточных методов, триангуляция Делоне задаёт структуру соседства через относительное положение точек, а не через принадлежность к абсолютным пространственным ячейкам. В двумерном случае строится триангуляция Делоне, а в трёхмерном — тетраэдрализация Делоне. Для статического множества точек в двумерной постановке построение выполняется за $O(n \log n)$, причём размер структуры линеен по числу вершин. В трёхмерном случае ситуация значительно сложнее: размер тетраэдрализации в худшем случае может достигать $O(n^2)$, что уже делает её менее удобной для крупных динамических расчётов. [13]

Ключевое достоинство Делоне-подхода состоит в том, что в двумерной динамической задаче при малых смещениях точек триангуляцию можно поддерживать локальными перестройками, например сериями переворотов рёбер. Поиск соседей фиксированного радиуса на основе триангуляции Делоне обычно выполняется как обход графа смежности, начиная с некоторой стартовой вершины.[14]

Методы на основе KNN. Методы данного класса исходно ориентированы не на фиксированный радиус, а на поиск k ближайших точек. Для поиска окрестности радиуса h сначала находится множество из k ближайших соседей, после чего оно отсекается по радиусу. Такой подход может быть работоспособным, если k выбирается с запасом относительно ожидаемого числа соседей, однако его корректность зависит от локальной плотности распределения частиц.

Kd-деревья, ball tree и похожие структуры обычно не очень удобны для частого динамического обновления. В таких случаях можно рассматривать приближенные методы, например Dynamic NN-Descent. Этот алгоритм строит граф k ближайших соседей, начиная с грубого набора соседей для каждой точки, а затем постепенно улучшает его, сравнивая точки с соседями их соседей. Он работает напрямую с текущим графом соседства и обновляет его по мере обработки, поэтому найденные связи могут использоваться сразу. В варианте Dynamic NN-Descent также уменьшается объём вспомогательных данных: кандидаты выбираются из уже имеющегося графа, а временные буферы остаются небольшими. Такой подход может быть полезен в динамических задачах, где точки или частицы постепенно перемещаются и нужно быстро поддерживать приближенную структуру соседства без полной перестройки. [15]

Навигационные графы. Навигационные графы малого мира, прежде всего HNSW, относятся к приближённым графовым методам поиска соседей. В таких структурах вершины соединяются рёбрами близости, а поиск выполняется жадной навигацией по графу. HNSW особенно важен тем, что является полностью графовым подходом и строится инкрементально, без необходимости в дополнительной грубой пространственной структуре. [16]

HNSW также является knn-алгоритмом. Большим его преимуществом является то, что списки ближайших соседей в нём уже сформированы на этапе перестройки графа, и он легко поддерживает динамическую структуру графа. Также он сам по себе является иерархическим, что улучшает его, как кандидата при использовании многоуровневых солверов, которые тоже разбивают облако точек на вложенные уровни. Но HNSW работает на ссылках, что даёт ему очень плохую кеш-локальность и параллелизацию на GPU. [16]

4.2 Обзор солверов для системы ограничений

Для задачи XPBI ключевыми требованиями к солверу являются устойчивость, предсказуемое время работы, возможность эффективного распараллеливания и приемлемая скорость сходимости на малом числе итераций.

Метод Якоби. Метод Якоби является одним из самых известных итеративных солверов. Ключевое свойство метода состоит в том, что все компоненты нового вектора вычисляются только по значениям предыдущей итерации - благодаря этому метод полностью параллелизуем. Но это приводит к значительно более медленному распространению информации по системе и, как следствие, требует большего числа итераций по сравнению с последовательными схемами.

Метод Гаусса–Зейделя. Метод Гаусса–Зейделя отличается от метода Якоби тем, что обновленные значения неизвестных немедленно используются при вычислении

следующих компонент. Такой способ обновления существенно ускоряет распространение локальных поправок по системе, что особенно важно для жёстких ограничений, которые важны для коллизий и твёрдых объектов. Поэтому метод Гаусса-Зейделя обычно сходится гораздо быстрее метода Якоби. Однако последовательная природа классического метода плохо согласуется с архитектурой GPU. Если ограничения зависят от общих степеней свободы, то одновременное обновление нескольких конфликтующих ограничений приводит к гонкам записи.

Один из наиболее распространенных способов распараллеливания метода Гаусса–Зейделя состоит в раскраске графа зависимостей. Пусть вершины графа соответствуют переменным, а ребро соединяет две вершины, если существует ограничение, связывающее их. Тогда задача состоит в разбиении вершин на цветовые классы так, чтобы внутри одного класса конфликтов не было; после этого ограничения одного цвета могут обрабатываться параллельно, а сами цвета — последовательно.

Двухступенчатый Метод Гаусса–Зейделя. Для устранения недостатков как чистого Якоби, так и чистого раскрашенного Гаусса–Зейделя применяются гибридные схемы. К этому классу относится Двухступенчатый Метод Гаусса–Зейделя (GS2), идея которого состоит в разделении процесса решения на два этапа: сначала выполняется небольшое число итераций Гаусса-Зейделя для быстрого распространения поправок, а затем — массивно параллельный этап уточнения. [5]

На первом этапе несколько итераций Гаусса-Зейделя дают качественное начальное приближение На втором этапе используется полностью параллельная схема, как правило Якоби или Якоби–Ричардсона, которая лучше загружает GPU и позволяет дешево выполнить большее число итераций. Такой двухэтапный алгоритм особенно полезен на GPU, когда чистый раскрашенный метод Гаусса-Зейделя недозагружает вычислительные блоки устройства. В результате достигается компромисс: по скорости сходимости метод превосходит чистый Якоби, а по степени параллелизма — чисто последовательные GS-схемы. [5]

Метод Чебышева. Полуитерационный метод Чебышева представляет собой способ ускорения простых итерационных схем, таких как метод Якоби, за счёт специального выбора весов на разных итерациях. Идея метода состоит в том, чтобы не просто многократно применять одно и то же обновление, а комбинировать последовательность итераций так, чтобы быстрее подавлять ошибку в решении. Важное свойство этого подхода состоит в том, что структура одной итерации остаётся параллельной, поэтому метод хорошо подходит для GPU. [17]

Нелинейные и квазиньютоновские методы. Более сложные методы, такие как Ньютон-Крылов и L-BFGS, пытаются учитывать не только локальную поправку текущей итерации, но и более полную информацию о форме всей нелинейной задачи. В этом смысле они движутся к решению более “глобально”: оценивают, в каком направлении следует изменять переменные, чтобы быстрее уменьшать невязку или энергию системы. За счёт этого такие методы нередко требуют меньше итераций до достижения высокой точности, чем простые локальные схемы. Однако каждая итерация оказывается существенно дороже, а сама процедура — сложнее в реализации и менее предсказуемой по времени выполнения, поэтому в задачах реального времени они обычно применяются реже.

Многоуровневые и иерархические методы. Отдельный класс эффективных солверов образуют многоуровневые методы, например, HPBD. Их основная идея заключается в том, что разные компоненты ошибки устраняются на разных уровнях разрешения: высокочастотная ошибка эффективно сглаживается локальными итерациями, а низкочастотная устраняется на более грубой сетке или подмножестве

элементов. [4]

Для позиционно-ориентированных методов это особенно естественно, поскольку грубый уровень можно интерпретировать как разреженное подмножество частиц, на котором быстро вычисляется приближенное решение, после чего оно переносится на полный уровень и уточняется локальными итерациями. Практически это означает, что даже сравнительно простой локальный метод, например раскрашенный метод Гаусса-Зейделя может работать существенно лучше, если ему предшествует иерархическая инициализация.

5 Построение решения задачи

5.1 Многоуровневый солвер

В соответствии с анализом в главе 4, в качестве первой ключевой оптимизации было выбрано использование иерархического солвера. Основной идеей является выделение из общего множества частиц вложенных подмножеств $L_{\max} \subset L_{\max-1} \subset \cdots \subset L_0$, называемых **уровнями**. Назовём нулевым уровнем L_0 всё множество частиц, и выберем некий коэффициент $k_l \in (0,1)$, такой что $|L_i| \approx k_l |L_{i-1}|$.

5.1.1 Перенос характеристик частиц

Для каждой частицы j на уровне L_0 найдём k_p ближайших частиц на уровне L_i, назовём их **родительскими вершинами частицы j уровня L_0 на уровне L_i**. Веса родителей нормированы и зависят от расстояния. Для каждой частицы уровня L_0 взвешенно распространим её ассоциированный объём и массу между всеми родителями на уровне L_i - эти суммы будут являться объёмом, ассоциированным с частицей грубого уровня.

При переходе к уровню L_i нельзя использовать текущую скорость вершины, так как она не отражает среднюю скорость кластера частиц, которые она обобщает в себе. Поэтому выполняется перенос скоростного остатка снизу вверх: для каждой вершины уровня L_0 вычисляется разность между её текущей скоростью и скоростью её родителей, после чего этот остаток взвешенно распределяется между ними. В результате грубый уровень получает усреднённую низкочастотную составляющую движения, которая действительно должна влиять на решение на данном масштабе.

5.1.2 Шаг солвера

Тогда на уровне L_i у каждой частицы есть позиция, скорость, масса и объём; деформационное состояние вычисляется по формуле (2). Тогда в рамках L_i формулируется задача XPBI, независимая от других уровней. Если эту задачу решить, мы получим грубое приближение решения исходной задачи на уровне L_0.

Пусть сделан шаг алгоритма на уровне L_i, и для каждой частицы получена коррекция скорости, тогда эту коррекцию можно распространить на уровень L_{i-1}. Для этого для каждой вершины уровня L_{i-1} определим её множество родительских вершин уровня L_i, и применим к ней их средневзвешенную коррекцию скорости.

Таким образом алгоритм выполняется поуровнево начиная от $L_{\max}$ до L_0, и каждый уровень в начале решения своей задачи уже имеет грубое приближение её решения, что может значительно ускорить сходимость решения задачи внутри уровня. При этом размеры уровней уменьшаются экспоненциально, что позволяет относительно быстро находить решение для них. Также иерархический солвер ускоряет распространение деформаций в системе: на высоких уровнях деформация влияет на удалённые друг от друга регионы облака точек, а затем распространяется обратно на исходное множество точек. [4]

5.1.3 Столкновения и закрепления

Жёсткие ограничения, привязанные к конкретным вершинам: столкновения и закреплённые частицы, также должны быть корректно обработаны на грубых уровнях. Для этого контакты сначала строятся на уровне L_0, а затем поднимаются на более грубые уровни через родительские вершины. Для каждого исходного контакта выбирается пара узлов, которая задаёт его с двух сторон, после чего эти узлы заменяются их

представителями на текущем уровне и по ним формируется укрупнённый контакт с той же нормалью, направлением действия и точкой приложения. Если на грубом уровне исходный контакт соответствует нескольким родительским вершинам, он представляется несколькими более слабыми контактами вместо одного, чтобы суммарное воздействие оставалось близким к контакту на уровне L_0. Далее такие контакты переводятся в обычный трафарет коллизионного ограничения текущего уровня и решаются вместе с остальными ограничениями этого уровня. Закреплённые вершины переносятся аналогично.

5.2 Поиск соседей

Солвер, описанный выше требует построения некоторой структуры поиска соседей на каждом уровне, причём факт того, что уровни вложены друг в друга потенциально может повлиять на её выбор. Стоит заметить, что при этом радиус, в котором запрашивается набор соседей, не фиксирован, и растёт с номером уровня. Поэтому, лучше всего для данной задачи в 2D подходят иерархия Делоне или HNSW: обе структуры нативно иерархические, работают в терминах относительного расположения частиц и допускают динамическое обновление. Из них была выбрана иерархия Делоне, поскольку, в отличие от HNSW, она является точной структурой, а также имеет регулярную структуру в памяти, что важно для GPU-реализации. [16]

5.2.1 Иерархия Делоне

Иерархия Делоне представляет собой последовательность вложенных триангуляций Делоне, построенных на вложенных подмножествах исходного множества точек. Поиск родительских вершин локализуется сверху вниз, причём на каждом уровне поиск стартует из вершины, найденной на более грубом уровне, что делает такую структуру естественно согласованной с многоуровневым солвером. Для поиска соседей делаются локальные обходы каждого уровня, являющегося триангуляцией Делоне. [6]

В физической симуляции структура облака частиц изменяется каждый шаг, но каждая отдельная частица смещается несильно относительно соседей. Для таких малых смещений триангуляцию можно исправлять локально в два этапа: сначала выполняется этап Fix, на котором устраняются инверсии треугольников, возникшие после сдвига вершин, затем этап Legalize, на котором последовательностью переворотов восстанавливается условие Делоне. Когда вершина за шаг пересекает не более одного ребра, триангуляцию можно исправить локально; если же вершина проходит через два или более рёбер, локальное исправление существенно усложняется — в основном разбиением сдвига вершины на несколько корректных. [14]

Для решения этой проблемы, раз в k_t шагов алгоритма, а также в начале симуляции, производится полная перестройка триангуляции на каждом уровне за $O(n \log n)$ в среднем алгоритмом Боуэра-Ватсона. [18]

5.2.2 Построение облака точек

Триангуляция Делоне наиболее стабильно работает на облаках точек без сильной кластеризации и больших пустот. Поэтому исходное облако точек строится модифицированным алгоритмом диска Пуассона:

В основе используется схема Бридсона: поддерживается список активных точек и фоновая сетка, а новые кандидаты генерируются в кольце радиусов $[R, 2R]$ вокруг уже принятых точек и принимаются только при соблюдении минимального расстояния R до ранее добавленных вершин. [19] Многоуровневый вариант строится сверху вниз: сначала

генерируется самый грубый уровень с большим радиусом запрета, затем последовательно добавляются новые точки для более тонких уровней с меньшими радиусами, при этом все вершины грубых уровней сохраняются. За счёт этого автоматически получается вложенная иерархия $L_{\max} \subset L_{\max -1} \subset \cdots \subset L_0$.

5.2.3 Запрос соседей в радиусе

Поиск соседей в радиусе в триангуляции Делоне обычно выполняется обходом в ширину от стартовой вершины. Был 50 раз проведён эксперимент на облаках по 10000 точек, и было установлено, что для облаков, сгенерированных методом диска Пуассона с радиусом R, примерно 95% вершин, находящихся на геометрическом расстоянии не более $2.5R$, одновременно находятся на рёберном расстоянии не более 2 от рассматриваемой вершины.

Поэтому далее был выбран радиус поддержки $R_{sup} = 2.5R$, а запрос всех соседей в геометрическом радиусе R_{sup} заменён эвристикой "запросить все вершины на рёберном расстоянии не более 2 с отсечением по R_{sup}". Такой выбор радиуса является подходящим, поскольку в этом радиусе в среднем находится порядка 18 вершин, что хорошо соотносится с оригинальным алгоритмом XPBI, где радиус выбирается так, чтобы у вершины в среднем было около 21 соседа. [1]

5.2.4 Параллелизация солвера

Поиск соседей также является важным компонентом параллелизации алгоритма раскрашенного Гаусса-Сейделя. Так как в рамках шага алгоритма вершина может обновлять состояние одного из своих соседей, при раскраске графа параллелизации вершина не должна иметь соседей данного цвета. Тогда эвристика, указанная выше, заменяет задачу раскраски плоскости на задачу раскраски графа триангуляции Делоне относительно рёберного расстояния 2.

Для триангуляции Делоне, построенной по облаку точек, построенных методом диска Пуассона, характерна достаточно регулярная структура и 6-валентность, что делает её близкой к регулярной треугольной сетке. [20] Для неё такая раскраска возможна в 7 цветов. [21] Эту раскраску мы используем как базовую и затем итеративно делаем несколько итераций разрешений конфликтов с повышением числа цветов до 16. [22] Был 50 раз проведён эксперимент на облаках по 10000 точек, и было установлено, что такой алгоритм за 2 итерации разрешает все кроме 1-2 конфликтов, которые обрабатываются отдельно.

5.2.5 3D случай

В данной статье в основном рассматривается 2D имплементация этого алгоритма. В трёхмерном случае использование тетраэдризации Делоне возможно, но является значительно менее эффективным и по построению, и по динамической перестройке. Поэтому в этом случае эффективно использовать либо HNSW, который работает одинаково в произвольной размерности, либо, как в оригинальном алгоритме, использовать клеточные методы с полной перестройкой каждый шаг. [16] [1]

5.3 Выбор солвера

В качестве солвера на каждом уровне иерархии был выбран двухэтапный метод Гаусса–Зейделя. Такой выбор обусловлен тем, что в рассматриваемой задаче необходимо одновременно сохранить хорошую локальную сходимость, характерную для

метода Гаусса–Зейделя, и обеспечить высокую степень параллелизма, необходимую для эффективной GPU-реализации. [5]

Основная идея алгоритма в том, что на каждом уровне сначала выполняется несколько проходов раскрашенного Гаусса–Зейделя, которые эффективно подавляют наиболее жёсткие локальные связи и сразу используют свежие обновления скоростей соседних частиц, быстро распространяя деформацию, а затем несколько более дешёвых и хорошо распараллеливаемых итераций второго этапа уточняют решение. Такая комбинация работает независимо с иерархическим переносом поправок между уровнями на каждом уровне. [5]

5.4 Псевдокод видоизменённого шага алгоритма XPBI

Обозначения. Пусть L_0 — исходное множество частиц, а $L_{\max} \subset \cdots \subset L_1 \subset L_0$ — последовательность грубых уровней. Через $\mathcal{T}_i$ обозначим триангуляцию Делоне уровня L_i, через χ_i — раскраску графа параллелизации уровня L_i.

Алгоритм 2 Один шаг многоуровневого XPBI-солвера

Входные данные: Состояние системы на шаге n, шаг времени Δt, иерархия уровней $\{L_i\}_{i=0}^{l_{\max}}$

Результат: Обновлённое состояние частиц уровня L_0

for $i = 0$ **to** $l_{\max}$ **do**
    Обновить триангуляцию Делоне $\mathcal{T}_i$ на уровне L_i ▷ локальным исправлением либо полной перестройкой
end for
Построить контакты и закрепления на уровне L_0
Применить внешние силы и получить предсказанные скорости
for $i = l_{\max}$ **downto** 0 **do**
    Найти для частиц уровня L_{i-1} их родительские вершины на уровне L_i
    Перенести скоростной остаток с уровня L_{i-1} на уровень L_i
    Перенести массу и объём с уровня L_{i-1} на уровень L_i
    Построить соседей для вершин уровня L_i по триангуляции $\mathcal{T}_i$ ▷ эвристика рёберного расстояния 2
    Вычислить матрицы коррекции градиента для частиц уровня L_i
    if $i > 0$ **then**
        Поднять контакты и закрепления с уровня L_0 на уровень L_i
        Построить по ним жёсткие ограничения уровня L_i
    else
        Построить жёсткие ограничения непосредственно на уровне L_0
    end if
    Обнулить множители Лагранжа ограничений
    Построить раскраску χ_i графа соседства уровня L_i
    Решить задачу XPBI на уровне L_i по алгоритму 3
    if $i > 0$ **then**
        Перенести найденную скоростную коррекцию с уровня L_i на уровень L_{i-1} ▷ Без учёта скоростного остатка, применённого в начале обработки
    end if
end for
Интегрировать положения частиц уровня L_0 с использованием полученных скоростей

Алгоритм 3 Решение задачи XPBI на уровне L_i

Входные данные: Уровень L_i, наборы соседей его вершин, раскраска графа соседства χ_i, набор ограничений, число итераций Гаусса–Зейделя $N_{GS}(i)$, число итераций Якоби-Ричардсона $N_{JR}(i)$

Результат: Обновлённые скорости частиц уровня L_i

for $s = 1$ **to** $N_{GS}(i)$ **do**
    for all цветов $c \in \chi_i$ **do**
        parallel for each частица $a \in c$
            Оценить градиент скорости частицы a по её соседям $\mathcal{N}(a)$
            Вычислить пробное деформационное состояние частицы a
            Выделить упругую часть деформации возвратным отображением
            Вычислить значение и градиенты материального ограничения частицы a
            Обновить множитель Лагранжа частицы a
            Обновить скорости частицы a и её соседей по алгоритму Гаусса–Зейделя
    end for
    Выполнить один проход решения жёстких ограничений уровня L_i
end for
for $s = 1$ **to** $N_{JR}(i)$ **do**
    parallel for each активная частица $a \in L_i$
        Оценить градиент скорости частицы a по её соседям $\mathcal{N}(a)$
        Вычислить пробное деформационное состояние частицы a
        Выделить упругую часть деформации возвратным отображением
        Вычислить значение и градиенты материального ограничения частицы a
        Обновить скорости частицы a и её соседей по алгоритму Якоби–Ричардсона
    Выполнить один проход решения жёстких ограничений уровня L_i
end for
for $s = 1$ **to** $N_{\text{pc}}(i)$ **do**
    Выполнить коррекцию положений частиц уровня L_i
end for
Выполнить XSPH-сглаживание скоростей на уровне L_i
if $i = 0$ **then**
    Зафиксировать новое деформационное состояние частиц уровня L_0
end if

6 Детали реализации

Реализация предложенного алгоритма доступна в репозитории GitHub [2].

В данной главе рассматриваются конкретные инженерные решения, использованные при реализации алгоритма, описанного в главе 5. Вся система реализована на связке `C#` и HLSL: CPU-часть отвечает за подготовку сцен, управление ресурсами, конфигурацией и порядком проходов, а GPU-часть выполняет все основные вычисления, связанные с поиском соседей, поддержкой иерархии, коллизиями и решением XPBI-задачи.

Симуляция организована как последовательность вычислительных проходов, записываемых CPU-частью в командный буфер и исполняемых на GPU. Такое разбиение позволяет отделить этапы подготовки данных, поддержки геометрических структур, построения ограничений и собственно решения системы, уменьшая связность отдельных модулей.

6.1 Организация данных

Все активные тела упаковываются в единый глобальный буфер. Для этого вершины всех объектов раскладываются в нём, а для каждого тела сохраняются его диапазоны и смещения. Такая организация позволяет запускать одни и те же проходы независимо от количества объектов и минимизирует число отдельных dispatch-вызовов. Иерархия уровней хранится внутри каждого тела в виде вложенных префиксов массива вершин. Префиксная таблица задаёт границы уровней и позволяет быстро получать число активных частиц на каждом из них.

Для каждой частицы хранятся положение, скорость, объём, масса, деформационное состояние, параметры материала и множители Лагранжа. Отдельно поддерживаются списки соседей фиксированной длины, отображения между глобальными и локальными индексами (две индексации поддерживаются в связи с объединением всех тел в один буфер), а также буферы родительских связей и весов. Грани триангуляции хранятся в виде наборов полурёбер со ссылками на противоположное ребро, что позволяет массивно параллелизовывать их обход на GPU. [23]

6.2 Поддержка геометрии и отношений соседства

Как было описано в главе 5, основная структура соседства строится на иерархии Делоне, поэтому в реализации на каждом уровне поддерживается собственная триангуляция. При этом триангуляция строится, хоть и на одном общем буфере, но без рёбер между физическими телами, так как такая триангуляция гораздо менее регулярна, чем триангуляция каждого тела по отдельности. На основе триангуляции строятся списки соседей, используемые солвером и раскраской.

Также для каждой вершины строятся в начале шага и кешируются её родительские вершины на следующем уровне и веса интерполяции. В реализации число родителей фиксировано и мало ($k_p = 2$), что позволяет избегать структур переменной длины. На GPU это важно для предсказуемого доступа к памяти и отсутствия дополнительных косвенных индексаций. Эти отображения используются для переноса массы и объёма на грубые уровни, переноса скоростного остатка и распространения найденной скоростной коррекции обратно на уровень ниже.

6.3 Набор ограничений

В реализации используются несколько типов ограничений, выполняющих разные функции в рамках одного шага решения. Основными являются материальные ограничения XPBI: они задают отклик материала через локальное деформационное состояние частицы, определяют вклад упругости, пластичности и объёмного поведения и именно они решаются в рамках GS2-солвера. [1]

Ограничения столкновений предотвращают взаимное проникновение тел и частиц, а ограничения закрепления фиксируют заданные вершины в пространстве или навязывают им предписанное движение; оба этих типа ограничений решаются отдельно от материальных, но в том же общем цикле итераций. Для построения ограничений столкновений контакты строятся только на уровне L_0 , после чего переводятся в трафареты ограничений. Для грубых уровней они поднимаются с уровня L_0 через родительские вершины, сохраняя тот же формат данных. Благодаря этому один и тот же проход решения жёстких ограничений используется на всех уровнях. Решение ограничений коллизий отделено от материальных, поэтому обе части можно настраивать независимо.

После основных итераций дополнительно применяются коррекция позиций и XSPH. Первая используется для устранения остаточных геометрических нарушений, прежде всего слишком малого расстояния между частицами и неустранённых пересечений, повышая устойчивость шага. Вторая выполняет сглаживание скоростей, подавляет высокочастотные осцилляции и уменьшает численный шум, не изменяя саму постановку материальной задачи. [1]

6.4 Организация солверов

Вычислительные проходы записываются в асинхронный командный буфер и исполняются отдельно от рендеринга. Для общих для рендеринга и физики буферов используется двойная буферизация: один слот читается рендерингом, второй записывается вычислениями физики. Синхронизация между этапами выполняется через барьеры синхронизации и контролируемую смену слотов. Это исключает конфликты чтения и записи между вычислениями и визуализацией, и позволяет выполнять их в различных графических очередях GPU, хотя добавляет задержку отображения на кадр.

7 Результаты

7.1 Конфигурация тестирования

Тестируемые алгоритмы. Для сравнения с оригинальным алгоритмом и выделения вклада отдельных оптимизаций тестируются 4 варианта алгоритма XPBI:

- Оригинальный XPBI
- XPBI с использованием многоуровневого солвера
- XPBI с использованием GS2-солвера
- XPBI со всеми предложенными оптимизациями

При этом все варианты используют триангуляцию Делоне как структуру поиска соседей для чистоты эксперимента. Сравнение триангуляции Делоне с клеточным алгоритмом производится отдельно.

Параметры тестирования. Тестирование производится в симуляциях с 10 000 активных частиц. Это число близко к целевой производительности для алгоритмов в реальном времени для игровых движков, и также именно на этом масштабе в оригинальной статье отмечены проблемы метода Гаусса–Зейделя. При использовании многоуровневого солвера, уровни строятся с коэффициентом $k_l = 0.133$, т.е. строится 4 уровня с размерами $[10\,000, 1\,330, 176, 23]$, и на всех уровнях, кроме L_0, делается ровно 1 итерация метода Гаусса–Зейделя. При этом параллельно происходит рендер симуляции в 30FPS. Симуляция производится на графическом ускорителе GeForce RTX 3080 ti с использованием технологий DirectX 12. Эта конфигурация является достаточно высокопроизводительной, но не самой новой из спектра целевых машин на которых может запускаться игра с использованием этого физического алгоритма. Для достоверности сравнения производительности алгоритмов они запускаются не с фиксированным числом итераций на уровне L_0, а оптимизируются до момента достижения некоторой фиксированной точности. Все результаты усреднены за 5000 шагов алгоритма в каждом из 20 независимых запусков.

Сценарий тестирования 1. Одно прямоугольное тело, состоящее из вязкого, тянущегося материала, подвешено за небольшие (порядка 20 частиц) кластеры точек за 2 верхних угла. В ходе симуляции оно прогибается и вытягивается вниз.

Сценарий тестирования 2. Два крупных тела, представляющих собой массу несмешивающихся жидкостей, падают на сложный пористый ландшафт

Методика сравнения. Для всех тестируемых вариантов измеряется число шагов симуляции в секунду при фиксированном ограничении на качество решения и при параллельном рендеринге сцены в 30 FPS. Критерием остановки итерационного процесса является достижение заданной точности по среднему значению ограничений после итераций, а не фиксированное число итераций. Такой способ сравнения позволяет сопоставлять именно вычислительную эффективность различных модификаций алгоритма, не смешивая её с влиянием различной степени сходимости.

Исследуемые метрики. В экспериментах анализируются следующие показатели:

- число шагов симуляции в секунду;
- число итераций, необходимых для достижения заданной точности;

- доля времени кадра, затрачиваемая на поиск соседей (вычисляется отдельно с помощью инструментария NVIDIA Nsight Graphics);
- качественное поведение материала, в частности скорость распространения деформации на большие расстояния и визуальная правдоподобность.

7.2 Результаты сравнения вариантов XPBI

Влияние многоуровневого солвера. Использование многоуровневой иерархии показало, что грубые уровни позволяют существенно ускорить распространение поправок по всему телу а также дают хорошее первое приближение решения на каждом шаге алгоритма. На рисунке 1 показана сходимость среднего значения ограничений в тестовом сценарии 1 с и без использования иерархического солвера: можно видеть, что значение ограничений в 0.3 $\sqrt{\text{Па}}$ достигается в среднем за вдвое меньшее число итераций. Дополнительные вычисления на грубых уровнях оказываются дешевыми из-за меньших размеров множеств L_i, поэтому суммарное время решения уменьшается. Помимо роста производительности, многоуровневый подход улучшает и качественное поведение симуляции: деформация распространяется на большие расстояния более естественно, без характерной для базового метода излишней локальности. Однако вклад этой оптимизации почти исчезает в сценах, где почти отсутствует однородность движения частиц - например, его вклад в сценарии 2 значительно меньше.

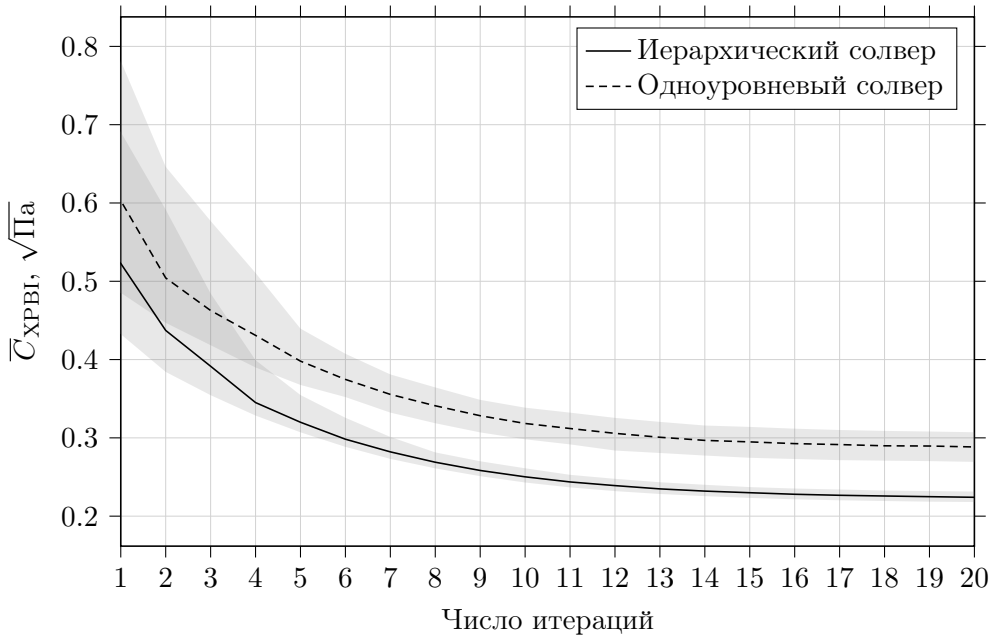


Рис. 1: Сходимость среднего остаточного XPBI-ограничения по числу итераций (медиана и интервал 25–75% для разных солверов)

Влияние GS2-солвера. Применение двухэтапного солвера GS2 также даёт положительный эффект по сравнению с исходным вариантом. Это связано с тем, что на рассматриваемом масштабе задачи раскрашенный метод Гаусса–Зейделя не всегда достаточно хорошо загружает GPU, особенно на поздних стадиях итерационного процесса. Комбинация небольшого числа итераций Гаусса–Зейделя и последующего блока итераций Якоби–Ричардсона позволяет сохранить приемлемую сходимость, одновременно лучше используя доступный параллелизм.

Совместное применение оптимизаций. Наилучшие результаты достигаются при одновременном использовании многоуровневого солвера и GS2. Эти оптимизации

воздействуют на разные аспекты вычислительного процесса: иерархия уменьшает число необходимых итераций за счёт лучшего начального приближения и переноса поправок с грубых уровней, а GS2 повышает эффективность выполнения самих итераций на GPU. В результате их совместное применение даёт наибольший выигрыш относительно исходного алгоритма.

Количественные результаты. В сценариях 1 и 2 были получены результаты, приведённые в табл. 1. Все измерения выполнены для симуляции из 10 000 частиц при одинаковой целевой точности решения и ограничении 30 FPS.

Метод	Шагов/сек. (сцен. 1)	Шагов/сек. (сцен. 2)
Исходный XPBI	310	280
XPBI с многоуровневым солвером	540	340
XPBI с GS2-солвером	450	410
XPBI со всеми оптимизациями	820	500

Таблица 1: Скорость работы различных вариантов XPBI

Из табл. 1 видно, что в тестовом сценарии 1 использование только многоуровневого солвера увеличивает быстродействие примерно в 1.74 раза, а использование только GS2 — примерно в 1.45 раза относительно исходного варианта. Совместное применение обеих оптимизаций даёт ускорение примерно в 2.6 раза, что является наилучшим результатом среди всех рассмотренных конфигураций. В сценарии 2 суммарное ускорение полного набора оптимизаций относительно исходной архитектуры составило около 1.8 раза. В данном сценарии вклад многоуровневой оптимизации значительно снижается, поскольку поведение системы в большей степени определяется локальными перестройками течения и взаимодействием с геометрически сложными препятствиями. Тем не менее даже в этих условиях предложенные модификации сохраняют заметное преимущество по производительности.

7.3 Результаты сравнения структур поиска соседей

На рисунке 2 показано итоговое распределение времени работы алгоритма по категориям. Сравнение с классическим алгоритмом показало, что использование динамически поддерживаемой триангуляции Делоне вместо клеточного алгоритма уменьшает вклад этапа поиска соседей в общее время работы симуляции с 14% до 5%. Это означает, что динамическая перестройка триангуляции, хоть и не отличается асимптотически от подхода в оригинальном алгоритме, даёт значительную константную оптимизацию. Помимо ускорения, триангуляция Делоне даёт и ряд практических преимуществ для интеграции алгоритма в игровой движок. Во-первых, она упрощает отрисовку внутренней структуры симулируемого объекта и визуальную отладку. Во-вторых, она позволяет явно поддерживать внешние границы объектов, что важно для прикладных задач, например, для навигации ИИ-агентов, анализа формы препятствий и построения вторичных физических или игровых представлений поверх результатов симуляции.



Рис. 2: Распределение времени работы алгоритма по категориям

7.4 Выводы по экспериментам

Таким образом, комбинация предложенных оптимизаций позволяет существенно приблизить XPBI к требованиям физической симуляции в реальном времени. На тестовых сценах с 10 000 частиц итоговый вариант алгоритма демонстрирует ускорение до 2.6 раза относительно исходной реализации, сохраняя при этом требуемую точность и обеспечивая более правдоподобное распространение деформаций.

8 Заключение

В работе была исследована задача ускорения алгоритма XPBI для моделирования деформируемых материалов в физическом движке реального времени. Для её решения была предложена модификация исходного метода, основанная на совместном использовании многоуровневого солвера, иерархической структуры соседства на базе триангуляции Делоне и двухэтапной схемы решения ограничений GS2, ориентированной на эффективное выполнение на GPU.

В ходе работы были проанализированы основные вычислительные узкие места исходного алгоритма XPBI, разработана новая схема шага симуляции, предложены способы переноса данных между уровнями и реализована полная GPU-версия алгоритма, доступная в репозитории GitHub [2]. Экспериментальное сравнение показало, что предложенные модификации позволяют существенно уменьшить стоимость шага симуляции при сохранении устойчивости вычислений, поддержки неупругих деформаций и корректной обработки контактов.

На тестовых сценариях с 10 000 частиц итоговый вариант алгоритма продемонстрировал ускорение до 2.6 раза по сравнению с исходной реализацией XPBI. При этом многоуровневый подход уменьшает число требуемых итераций за счёт более быстрого распространения поправок по телу, а GS2-солвер повышает эффективность использования вычислительных ресурсов GPU. Дополнительно показано, что использование иерархии Делоне снижает относительную стоимость поиска соседей и даёт практические преимущества для интеграции в игровой движок.

Таким образом, поставленная цель работы достигнута: разработана и исследована модификация XPBI, обеспечивающая более высокую производительность при сохранении качественного поведения симуляции. Полученные результаты показывают, что предложенный подход является перспективным для применения в интерактивных физических системах и может служить основой для дальнейшего развития метода, в том числе для трёхмерного случая и более сложных сценариев взаимодействия материалов.

Список литературы

[1] Position-Based Dynamics Handles Updated Lagrangian Inelasticity / Chang Yu, Xuan Li, Lei Lan et al. // *arXiv preprint*. — 2024.

[2] *Кирьянов, Григорий*. Реализация улучшенного алгоритма XPBI. — Репозиторий GitHub. `https://github.com/SingularGamesStudio/Shattered`.

[3] Position based dynamics / Matthias Müller, Bruno Heidelberger, Marcus Hennix, John Ratcliff // *Journal of Visual Communication and Image Representation*. — 2007.

[4] *Müller, Matthias*. Hierarchical Position Based Dynamics / Matthias Müller // Workshop on Virtual Reality Interactions and Physical Simulations. — 2008.

[5] Two-Stage Gauss–Seidel Preconditioners and Smoothers for Krylov Solvers on a GPU Cluster / Luc Berger-Vergiat, Christian Glusa, Mark Ainsworth, Mark T. Jones // *arXiv preprint*. — 2021. — Vol. arXiv:2104.01196.

[6] *Devillers, Olivier*. The Delaunay Hierarchy / Olivier Devillers // *International Journal of Foundations of Computer Science*. — 2002. — Vol. 13, no. 2.

[7] *Macklin, Miles*. XPBD: Position-Based Simulation of Compliant Constrained Dynamics / Miles Macklin, Matthias Müller, Nuttapong Chentanez // Proceedings of the 9th International Conference on Motion in Games. — 2016.

[8] *Sulsky, Deborah L.* Application of a particle-in-cell method to solid mechanics / Deborah L. Sulsky, Shi-Jian Zhou, Howard L. Schreyer // *Computer Physics Communications*. — 1995. — Vol. 87.

[9] *Bouaziz, Sofien et al.* Projective Dynamics: Fusing Constraint Projections for Fast Simulation / Sofien et al. Bouaziz // *ACM Transactions on Graphics*. — 2014.

[10] Vertex Block Descent / Anka He Chen, Ziheng Liu, Yin Yang, Cem Yuksel // *ACM Transactions on Graphics*. — 2024.

[11] *Monaghan, J. J.* On the problem of penetration in particle methods / J. J. Monaghan // *Journal of Computational Physics*. — 1989. — Vol. 82.

[12] *Chialvo, Ariel A.* On the use of the Verlet neighbor list in molecular dynamics / Ariel A. Chialvo, Pablo G. Debenedetti // *Computer Physics Communications*. — 1990. — Vol. 60.

[13] *Delaunay, Boris*. Sur la sphère vide / Boris Delaunay // *Bulletin de l'Académie des Sciences de l'URSS, Classe des sciences mathématiques et naturelles*. — 1934.

[14] Maintaining 2D Delaunay triangulations on the GPU for proximity queries of moving points / Heinich Porro, Benoît Crespin, Nancy Hitschfeld et al. // SIAM International Meshing Roundtable Workshop. — 2023.

[15] Dynamic NN-Descent: An Efficient k-NN Graph Construction Method / Jie-Feng Wang, Wan-Lei Zhao, Shihai Xiao et al. // *arXiv preprint*. — 2024.

[16] *Malkov, Yu. A.* Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs / Yu. A. Malkov, D. A. Yashunin // *arXiv preprint*. — 2016.

[17] *Varga, Richard S.* On Hybrid Semi-Iterative Methods / Richard S. Varga // *Numerische Mathematik*. — 1959.

[18] *Watson, David F.* Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes / David F. Watson // *The Computer Journal*. — 1981. — Vol. 24, no. 2.

[19] *Bridson, Robert.* Fast Poisson Disk Sampling in Arbitrary Dimensions / Robert Bridson // SIGGRAPH Sketches. — 2007.

[20] Efficient and good Delaunay meshes from random points / Mohamed S. Ebeida, Scott A. Mitchell, Andrew A. Davidson et al. // *Computer-Aided Design*. — 2011. — Vol. 43, no. 11.

[21] Simpler Multicoloring of Triangle-free Hexagonal Graphs // *Discrete Mathematics*. — 2011.

[22] *Chen, Xuhao.* Efficient and High-quality Sparse Graph Coloring on the GPU / Xuhao Chen, Pingfan Li, Canqun Yang // *arXiv preprint*. — 2016.

[23] *Campagna, Swen.* Directed Edges: A Scalable Representation For Triangle Meshes / Swen Campagna, Leif Kobbelt, Hans-Peter Seidel // *Journal of Graphics Tools*. — 1998. — Vol. 3, no. 4.